Università degli Studi di Salerno

Corso di Ingegneria del Software

TrackFit
ODD_TrackFit
Versione 1.0

TrackFit
Sistema di Tracking
Allenamenti

Data: 02/03/2026

Coordinatore del progetto:

Nome	Matricola
Gabriele di Palma	0512119257

Partecipanti:

Nome	Matricola
Yuri Fierro	0512119524

Scritto da:	Gabriele di Palma

Revision History

Data	Versione	Descrizione	Autore
02/03/2026	1.0	Prima stesura dell'ODD	GDP, YF

Indice

1. INTRODUZIONE 4
1.1. Purpouse of the system 4
1.2. Design objectives 4
1.3. Object model optimization 5
2. PACKAGES 5
2.1. Project structure 5
3. CLASS INTERFACES 6
3.1 UserManagement 6
3.2 WorkoutManagement 8
3.3 SubscriptionManagement 10
3.4 CoachManagement 12
3.5 GymsManagement 14
3.6 ReportManagement 15
4. DESIGN PATTERNS 17
4.1 Pattern dei servizi 17
4.2 Pattern dei dati persistenti 17
4.3 Pattern per le notifiche 17
4.4 Pattern per l'interoperabilità 18
4.5 Pattern per la gestione dello stato offline 18

1. INTRODUZIONE

1.1. Purpose of the system

TrackFit si propone come una piattaforma digitale integrata per il monitoraggio e l'analisi delle attività sportive, rivolta a utenti amatoriali e semi-professionisti. Il sistema consente la registrazione personalizzata delle sessioni di allenamento, la gestione dei progressi attraverso dashboard interattive, la prenotazione di allenatori certificati e la gestione completa degli abbonamenti alle palestre partner.

Il portale, accessibile sia tramite interfaccia web che app mobile, gestisce quattro categorie di utenti: Atleta, Coach, Gestore della Palestra (Amministratore di struttura) e Host (Super-amministratore di sistema). Ogni ruolo dispone di funzionalità dedicate e di un pannello di controllo personalizzato.

1.2. Design objectives

Il sistema dovrà essere progettato in modo da garantire:

- Usabilità
  - L'interfaccia utente deve essere responsiva, intuitiva e coerente su tutte le piattaforme (web, tablet, mobile). Tutti gli input devono essere validati in tempo reale, con feedback immediato all'utente.
- Affidabilità
  - Nessun errore deve rimanere ingestito. Le sessioni di allenamento devono essere salvate localmente in caso di assenza di rete e sincronizzate non appena la connessione è ripristinata. Il sistema deve garantire un uptime del 99,5%.
- Sicurezza
  - Tutte le comunicazioni avvengono tramite HTTPS. Le password sono memorizzate con algoritmo di hashing robusto (bcrypt). L'accesso alle risorse è regolato da un sistema RBAC (Role-Based Access Control). I pagamenti rispettano lo standard PCI-DSS tramite integrazione con gateway certificato (es. Stripe).
- Scalabilità
  - L'architettura cloud deve supportare picchi di carico simultanei (es. fine gara con migliaia di atleti che caricano dati contemporaneamente) mediante auto-scaling orizzontale.
- Interoperabilità
  - Il sistema deve supportare l'importazione di file GPX/TCX e l'integrazione con API di terze parti (Strava, Google Fit) tramite protocollo OAuth 2.0, oltre all'esportazione in formati CSV e PDF.

1.3. Object model optimization

Rispetto al modello definito in fase di analisi (RAD v1.2), l'Object Model è stato ottimizzato con le seguenti modifiche principali:

- La classe Workout acquisisce l'attributo stato (Pianificata, InCorso, CompletataLocalmente, InSincronizzazione, Consolidata) per modellare l'intero ciclo di vita di una sessione, incluso il recupero dopo crash del dispositivo.
- La classe Abbonamento acquisisce l'attributo qrCode e il metodo generaQRCode(), necessari per il caso d'uso UC9 (Accesso Fisico alla Struttura).
- La classe Coupon viene aggiunta al modello per supportare la logica di sconto (UC8), con attributi codice, percentualeSconto, validita, rimosso e il metodo applicaSconto().
- La classe Pagamento viene aggiunta con attributi idPagamento, importo, dataPagamento, stato e metodi eseguiPagamento() e generaFattura(), necessari per i casi d'uso UC7 e UC8.
- La classe ListaAttesa viene introdotta per modellare la coda degli utenti in attesa di un posto in un corso pieno (FR25), con metodi aggiungiUtente() e notificaUtente().
- Workout acquisisce un metodo recuperaDati() per gestire il ripristino delle sessioni dopo spegnimento imprevisto del dispositivo (UC4 alternativa).

2. PACKAGES

2.1. Project structure

La struttura del progetto si presenta nel modo seguente:

- **Frontend** (Next.js / TypeScript)
  - src/app — routing e pagine Next.js (App Router)
  - src/components — componenti React riutilizzabili
  - src/hooks — custom React hooks
  - src/store — stato globale (es. Zustand)
  - src/styles — CSS Modules / Tailwind
- **Backend** (Next.js API Routes / Node.js)
  - src/app/api — endpoint RESTful
  - src/lib — logica di business e servizi
  - src/db — accesso al database (MySQL)

Il backend è suddiviso in moduli applicativi secondo una separazione in sottosistemi. I servizi principali sono organizzati nei seguenti package:

Package	Responsabilità
UserManagement	Registrazione, autenticazione (JWT), gestione profili e ruoli (RBAC)
WorkoutManagement	Pianificazione, tracciamento live, salvataggio locale e sincronizzazione delle sessioni
GymsManagement	Creazione e configurazione di palestre, corsi e orari
SubscriptionManagement	Gestione abbonamenti, pagamenti, coupon, fatture e accesso QR
CoachManagement	Gestione del roster atleti, modifica piani di allenamento, vincoli di preavviso
ReportManagement	Generazione di report PDF/CSV per utenti, coach e gestori
NotificationService	Notifiche push, email, promemoria e avvisi di lista d'attesa
AuditLog	Registrazione di tutte le operazioni amministrative con timestamp e utente

Ciascun modulo di management adotta internamente la seguente struttura a layer:

- domain — classi di dominio, enumeratori e logica di business
- application — casi d'uso (port/in e port/out) e service
- infrastructure — entity JPA/Prisma, repository, mapper
- presentation — controller REST, oggetti request/response, mapper di presentazione

3. CLASS INTERFACES

Di seguito sono riportate le interfacce, i vincoli OCL e le descrizioni per ciascun modulo di sistema.

3.1. UserManagement

UserManagement
+login(LoginRequest request) : User +register(RegisterRequest request) : User +updateProfile(UpdateProfileRequest request) : void
+recoverPassword(String email) : void
+deleteAccount(String userId) : void

Constraints:

context UserService::login(email: String, password: String) : User
pre:
email <> null and not email.isEmpty()
and email.matches('[^@\\s]+@[^@\\s]+\\.[^@\\s]+')
and password <> null and not password.isEmpty()
and Database.users->exists(u | u.email = email and u.passwordHash = hash(password))
post:
result = Database.users->select(u | u.email = email)->first()
and result.role <> null

Descrizione: Autentica un utente verificando le credenziali e restituisce l'account con il ruolo associato. In caso di credenziali errate esegue UC12.

context UserService::register(name, surname, email, username, password: String) : User
pre:
name <> null and not name.isEmpty()
and email <> null and email.matches('[^@\\s]+@[^@\\s]+\\.[^@\\s]+')
and username <> null and not username.isEmpty()
and password <> null and not password.isEmpty()
and not Database.users->exists(u | u.email = email)
and not Database.users->exists(u | u.username = username)
post:
Database.users->exists(u | u.email = email and u.username = username)
and result.email = email and result.username = username

Descrizione: Registra un nuovo utente nel sistema assegnandogli il ruolo di default Atleta. L'email e lo username devono essere univoci.

context UserService::recoverPassword(email: String) : void
pre:
email <> null and not email.isEmpty()
post:
if Database.users->exists(u | u.email = email) then
ServizioEmail.inviato(email, token) = true

```
else
  -- avviso generico per sicurezza (no info leak)
endif
```

Descrizione: Avvia la procedura di recupero password inviando un token temporaneo via email. Se l'email non esiste, il sistema mostra un avviso generico per ragioni di sicurezza (UC12).

context UserService::deleteAccount(userId: String) : void

```
pre:
  userId <> null and not userId.isEmpty()
  and Database.users->exists(u | u.id = userId)

post:
  not Database.users->exists(u | u.id = userId)
  and Database.workouts->forAll(w | w.user.id <> userId)
  -- Conformità GDPR: tutti i dati personali rimossi
```

Descrizione: Cancellazione permanente dell'account e di tutti i dati associati, in conformità con il GDPR (FR30).

3.2. WorkoutManagement

WorkoutManagement

```
+pianificaSessione(PianificaSessioneRequest request) : Workout
+avviaTracciamento(String workoutId) : void
+aggiornaMetricheLive(AggiornaMetricheRequest request) : void
+terminaSessione(TerminaSessioneRequest request) : Workout
+recuperaSessione(String workoutId) : Workout
+sincronizzaSessione(String workoutId) : void
+getStoricoAllenamenti(String userId) : List<Workout>
+getSessioneById(String workoutId) : Workout
```

Constraints:

context WorkoutService::pianificaSessione(userId, tipo, dataOra, durata, obiettivo, sensori: ...)
pre:
userId <> null and Database.users->exists(u | u.id = userId)
and tipo <> null and not tipo.isEmpty()
and dataOra <> null and dataOra > LocalDateTime::now()
and durata > 0
post:
let w = Workout.allInstances()->any(w | w.user.id = userId and w.dataOra = dataOra) in
w <> null and w.stato = WorkoutStatoEnum::PIANIFICATA
and ServizioNotifiche.promemoriaProgrammato(userId, dataOra - 10min) = true

Descrizione: Pianifica una nuova sessione di allenamento per l'utente. Il sistema programma automaticamente una notifica push 10 minuti prima dell'inizio (UC3).

context WorkoutService::terminaSessione(workoutId: String, percezione, note: ...) : Workout
pre:
workoutId <> null and not workoutId.isEmpty()
and Database.workouts->exists(w | w.id = workoutId and w.stato = WorkoutStatoEnum::IN_CORSO)
post:
let w = Database.workouts->any(w | w.id = workoutId) in
w.stato = WorkoutStatoEnum::COMPLETATA_LOCALMENTE
and w.percezionesSforzo = percezione
and StorageLocale.salvato(workoutId) = true

Descrizione: Termina una sessione attiva, calcola il riepilogo e salva i dati in locale in attesa di sincronizzazione (UC4).

context WorkoutService::recuperaSessione(workoutId: String) : Workout
pre:
workoutId <> null
and StorageLocale.exists(workoutId) = true
and StorageLocale.anomaliaRilevata(workoutId) = true
post:

```
let w = StorageLocale.carica(workoutId) in
  w <> null and w.stato = WorkoutStatoEnum::IN_CORSO
-- Il tracciamento riprende dal punto di salvataggio
```

Descrizione: Recupera una sessione interrotta a seguito di crash del dispositivo (es. batteria esaurita), ripristinando i dati salvati nel file temporaneo (UC4 alternativa).

```
context WorkoutService::sincronizzaSessione(workoutId: String) : void
pre:
  workoutId <> null
  and StorageLocale.exists(workoutId) = true
  and GestoreRete.connessioneDisponibile() = true
  and Database.workouts->forAll(w | w.id <> workoutId)

post:
  Database.workouts->exists(w | w.id = workoutId)
  and let w = Database.workouts->any(w | w.id = workoutId) in
    w.stato = WorkoutStatoEnum::CONSOLIDATA
  and Dashboard.obiettivi->aggiornati = true
```

Descrizione: Invia la sessione salvata in locale al backend cloud, verifica l'assenza di duplicati e marca la sessione come Consolidata. Aggiorna lo stato degli obiettivi nella dashboard (UC5).

3.3. SubscriptionManagement

SubscriptionManagement
+acquistaAbbonamento(AcquistoAbbonamentoRequest request) : Abbonamento
+applicaCoupon(String codice, String abbonamentoId) : Double
+validaAccesso(String qrCode, String strutturaId) : Boolean
+cancellaPrenotazione(String prenotazioneId) : void
+getStatoAbbonamento(String userId) : Abbonamento
+rinnova(String abbonamentoId) : Abbonamento

Constraints:

context SubscriptionService::acquistaAbbonamento(userId, tipoId, couponCode: ...) : Abbonamento
pre:
userId <> null and Database.users->exists(u | u.id = userId)
and tipoId <> null and Database.tipiAbbonamento->exists(t | t.id = tipoId)
and (couponCode = null or Database.coupon->exists(c | c.codice = couponCode
and c.validita > now() and not c.rimosso))
post:
Database.abbonamenti->exists(a | a.user.id = userId and a.stato = 'Attivo')
and let a = Database.abbonamenti->any(a | a.user.id = userId) in
a.qrCode <> null
and if couponCode <> null then
Database.coupon->any(c | c.codice = couponCode).rimosso = true
endif
and Database.fatture->exists(f | f.abbonamento.user.id = userId)

Descrizione: Attiva un abbonamento per un utente, applicando eventuale coupon sconto. Genera QR code, fattura PDF e registra nel log di audit (UC8).

context SubscriptionService::validaAccesso(qrCode: String, strutturaId: String) : Boolean
pre:
qrCode <> null and not qrCode.isEmpty()
and strutturaId <> null
and Database.strutture->exists(s | s.id = strutturaId)
post:
let a = Database.abbonamenti->any(ab | ab.qrCode = qrCode
and ab.struttura.id = strutturaId) in
if a <> null and a.stato = 'Attivo' and a.dataFine > now() then
result = true
and Database.accessi->exists(acc | acc.qrCode = qrCode and acc.timestamp = now())
else
result = false
endif

Descrizione: Verifica la validità del QR code ai tornelli della struttura. Se l'abbonamento è attivo, registra il check-in e sblocca l'accesso. In caso contrario, nega l'ingresso (UC9).

context SubscriptionService::cancellaPrenotazione(prenotazioneId: String) : void
pre:
prenotazioneId <> null
and Database.prenotazioni->exists(p | p.id = prenotazioneId) and let p = Database.prenotazioni->any(p | p.id = prenotazioneId) in
p.stato = 'Confermata'
post:
let p = Database.prenotazioni->any(p | p.id = prenotazioneId) in
if p.dataPrenotazione - now() >= 24h then
p.stato = 'Cancellata' and rimborso = importoTotale else
p.stato = 'Cancellata' and rimborso = importoTotale * 0.5
endif -- Vincolo R3: cancellazione gratuita >= 24h, penale 50% oltre il limite

Descrizione: Cancella una prenotazione confermata applicando la politica di rimborso: cancellazione gratuita se effettuata almeno 24 ore prima, penale del 50% altrimenti (Vincolo di dominio R3).

3.4. CoachManagement

CoachManagement
+prenotaSlotCoach(PrenotazioneCoachRequest request) : Prenotazione +modificaPianoAtleta(ModificaPianoRequest request) : void +getCalendarioAtleta(String atletaId) : List<Workout> +getRosterAtleti(String coachId) : List<Atleta> +getReportAtleta(String atletaId) : Report

Constraints:

context CoachService::prenotaSlotCoach(userId, coachId, dataOra: ...) : Prenotazione
pre:
userId <> null and Database.users->exists(u | u.id = userId) and coachId <> null and Database.coaches->exists(c | c.id = coachId) and dataOra > LocalDateTime::now()
and not Database.prenotazioni->exists(p | p.coach.id = coachId

```
    and p.dataOra = dataOra and p.stato = 'Confermata')

post:
  Database.prenotazioni->exists(p | p.user.id = userId and p.coach.id = coachId
    and p.stato = 'Confermata')
  and ServizioEmail.inviato(userId, 'conferma_prenotazione') = true
  and ServizioEmail.inviato(coachId, 'nuova_prenotazione') = true
  and Database.fatture->exists(f | f.prenotazione.user.id = userId)
```

Descrizione: Prenota uno slot disponibile con un coach, elabora il pagamento tramite gateway esterno (SCA 3-D Secure), genera ricevuta PDF e blocca i calendari di utente e coach (UC7).

```
context CoachService::modificaPianoAtleta(coachId, atletaId, sessioneId, nuovaDataOra,
motivazione: ...) : void
pre:
  coachId <> null and Database.coaches->exists(c | c.id = coachId)
  and atletaId <> null and Database.users->exists(u | u.id = atletaId)
  and sessioneId <> null and Database.workouts->exists(w | w.id = sessioneId)
  and nuovaDataOra > LocalDateTime::now()
  and motivazione <> null and not motivazione.isEmpty()
  and let sessione = Database.workouts->any(w | w.id = sessioneId) in
    (sessione.dataOra - LocalDateTime::now()) >= Duration.ofHours(48)
  -- Vincolo R1: preavviso minimo 48 ore

post:
  let w = Database.workouts->any(w | w.id = sessioneId) in
    w.dataOra = nuovaDataOra
  and ServizioEmail.inviato(atletaId, riepilogo_modifica) = true
  and AuditLog.registrato(coachId, sessioneId, nuovaDataOra, motivazione) = true
```

Descrizione: Consente al coach di modificare l'orario di una sessione futura di un atleta. Il sistema verifica il rispetto del preavviso minimo di 48 ore (Vincolo R1). In caso di violazione, esegue UC15. Invia email automatica all'atleta e registra l'evento nel log di audit (UC6).

3.5. GymsManagement

GymsManagement
+creaStruttura(CreaStrutturaRequest request) : Struttura +aggiungiCoach(String strutturaId, String email) : void
+creaCorsoPalestra(CreaCorsoPalestraRequest request) : Corso
+prenotaCorsoPalestra(String userId, String corsoId) : Prenotazione
+aggiungiListaAttesa(String userId, String corsoId) : void
+getStruttureByGestore(String gestoreId) : List<Struttura>
+rimuoviStruttura(String strutturaId) : void

Constraints:

context GymsService::creaStruttura(piva, cun, denominazione, indirizzo: String) : Struttura
pre: piva <> null and not piva.isEmpty()
and cun <> null and not cun.isEmpty()
and denominazione <> null and not denominazione.isEmpty() and not Database.strutture->exists(s | s.piva = piva)
and not Database.strutture->exists(s | s.cun = cun)
and not Database.strutture->exists(s | fuzzyMatch(s.denominazione + s.indirizzo,
denominazione + indirizzo) > 0.9)
-- Vincoli R8 (P.IVA e CUN univoci) e R9 (anti-duplicato fuzzy)
post: Database.strutture->exists(s | s.piva = piva and s.cun = cun and s.stato = 'Attiva')
and AuditLog.registrato('creazione_struttura', piva, now()) = true
and ServizioEmail.inviato(adminEmail, 'struttura_creata') = true

Descrizione: Crea una nuova struttura palestra verificando l'univocità di P.IVA e CUN e l'assenza di duplicati tramite algoritmo di similarità fuzzy su denominazione e indirizzo (Vincoli R8, R9). Registra l'operazione nell'audit log (UC2).

context GymsService::prenotaCorsoPalestra(userId: String, corsoId: String) : Prenotazione
pre: userId <> null and Database.users->exists(u | u.id = userId)
and corsoId <> null and Database.corsi->exists(c | c.id = corsoId)

```
  and let abbonamento = Database.abbonamenti->any(a | a.user.id = userId and a.stato = 'Attivo') in
    abbonamento <> null
  and let corso = Database.corsi->any(c | c.id = corsoId) in
    corso.postiOccupati < corso.capacitaMassima

post:
  Database.prenotazioni->exists(p | p.user.id = userId and p.corso.id = corsoId
    and p.stato = 'Confermata')
  and let corso = Database.corsi->any(c | c.id = corsoId) in
    corso.postiOccupati = corso.postiOccupati@pre + 1
```

Descrizione: Prenota un posto in un corso, verificando che l'utente abbia un abbonamento attivo e che il corso abbia posti disponibili. Se il corso è pieno, l'utente può iscriversi alla lista d'attesa tramite aggiungiListaAttesa (FR25).

3.6. ReportManagement

ReportManagement
+generaReportUtente(String userId, String periodo, String tipo) : Report
+generaReportCoach(String coachId, String atletaId) : Report
+generaReportGestore(String strutturaId, String periodo) : Report
+esportaCSV(String reportId) : File
+esportaPDF(String reportId) : File
+importaGPX(String userId, File file) : Workout
+esportaGPX(String workoutId) : File

Constraints:

context ReportService::generaReportUtente(userId, periodo, tipo: String) : Report

```
pre:
  userId <> null and Database.users->exists(u | u.id = userId)
  and periodo <> null and not periodo.isEmpty()
  and tipo <> null and not tipo.isEmpty()
  and Database.workouts->select(w | w.user.id = userId and w.stato = 'CONSOLIDATA'
      and w.dataOra.inPeriodo(periodo))->size() > 0
```

```
post:
  let r = Report.allInstances()->any(r | r.user.id = userId and r.periodo = periodo) in
    r <> null
    and r.formato = 'PDF'
    and r.distanzaTotale = Database.workouts->select(w | w.user.id = userId
        and w.stato = 'CONSOLIDATA' and w.dataOra.inPeriodo(periodo)).distanza->sum()
```

Descrizione: Genera un report aggregato per l'utente nel periodo selezionato, includendo distanza totale, tempo, ritmo medio e andamento settimanale. Se non vi sono dati nel periodo, il sistema inibisce la generazione (UC11).

```
context ReportService::importaGPX(userId: String, file: File) : Workout
pre:
  userId <> null and Database.users->exists(u | u.id = userId)
  and file <> null and (file.estensione = 'gpx' or file.estensione = 'tcx')

post:
  let w = Workout.allInstances()->any(w | w.user.id = userId and w.sorgente = 'IMPORT') in
    w <> null
    and not Database.workouts->exists(w2 | w2.user.id = userId
        and w2.dataOra = w.dataOra and w2.id <> w.id)
  -- Verifica anti-duplicato basata su dataOra e utente
```

Descrizione: Importa una sessione di allenamento da file GPX o TCX, effettuando il parsing lato server e verificando l'assenza di duplicati (FR27).

4. DESIGN PATTERNS

4.1. Object model optimization

All'interno dell'applicazione viene adottato un design pattern ibrido che combina il Facade Pattern con l'architettura Ports & Adapters (Hexagonal Architecture).

Ogni modulo espone le proprie funzionalità esclusivamente attraverso interfacce di porta (port/in per i casi d'uso in ingresso, port/out per i servizi in uscita). Le implementazioni concrete sono incapsulate nelle classi ManagerService, rendendo il sistema indipendente dai dettagli tecnologici (database, API esterne, gateway di pagamento).

Per ogni entità di dominio principale sono identificati quattro servizi corrispondenti alle operazioni CRUD:

- CreateManagerService — gestisce la creazione dell'entità
- ReadManagerService — gestisce la lettura e il recupero dell'entità
- UpdateManagerService — gestisce l'aggiornamento dell'entità
- DeleteManagerService — gestisce la rimozione dell'entità

Questo approccio garantisce la separazione delle responsabilità (SRP), facilita il testing unitario tramite mock delle porte e permette la sostituzione delle implementazioni senza impattare il dominio.

4.2. Pattern dei dati persistenti

Per la persistenza dei dati viene adottato il Repository Pattern, che funge da astrazione tra la logica di dominio e il layer di accesso ai dati. Nell'implementazione specifica:

- Lato backend Node.js/TypeScript: viene utilizzato Prisma ORM con MySQL, che implementa il pattern Repository attraverso le Prisma Client APIs.
- Per l'app mobile (React Native / PWA): viene utilizzato AsyncStorage per la persistenza locale delle sessioni in corso, garantendo il funzionamento offline e il recupero dati in caso di crash (UC14).

Ogni Repository è definito come interfaccia nel layer domain e implementato nel layer infrastructure, seguendo il principio di inversione delle dipendenze (DIP).

4.3. Pattern per le notifiche

Il sistema di notifiche (push, email, in-app) adotta il pattern Observer. Il NotificationService è il soggetto osservato; i canali di notifica (Firebase Cloud Messaging, Servizio Email) sono gli osservatori registrati. Questo consente di aggiungere nuovi canali senza modificare la logica di business.

Esempi di eventi che scatenano notifiche:

- Promemoria sessione programmata (10 minuti prima dell'inizio)

- Modifica piano allenamento da parte del coach
- Conferma prenotazione corso o slot allenatore
- Disponibilità di un posto in lista d'attesa
- Cancellazione di un corso da parte del gestore

4.4. Pattern per l'interoperabilità

Per l'integrazione con servizi esterni (Strava, Google Fit, gateway di pagamento Stripe) viene adottato il pattern Adapter. Ogni integrazione esterna è incapsulata in un adapter che traduce le chiamate del dominio nel formato richiesto dall'API di terze parti, utilizzando il protocollo OAuth 2.0 per l'autenticazione.

Questo approccio consente di sostituire un provider esterno (es. cambiare gateway di pagamento) modificando esclusivamente il relativo adapter, senza impattare la logica di business.

4.5. Pattern per la gestione dello stato offline

La gestione del ciclo di vita delle sessioni di allenamento è modellata attraverso il pattern State. I possibili stati di un Workout (Pianificata, InCorso, Interrotta, InPausa, CompletataLocalmente, InAttesaDiRete, InSincronizzazione, Consolidata) definiscono le transizioni valide e le azioni permesse in ciascuno stato.

La strategia di sincronizzazione (Wi-Fi vs. rete cellulare, priorità della coda) è implementata tramite il pattern Strategy, permettendo di adattare il comportamento in base al tipo di connessione disponibile.